



Soundstream

Mise à jour synapse - Analyse

Préparé par :

FILLONNEAU Kenzo
HEURTEAUX Romain
SEBBAH Sofiane
HOARAU Lucas
LEMEST Enzo

18 juin 2026

Table des matières

1	Introduction	2
2	Analyse	3
2.1	Fonctionnalités existantes	3
2.2	Axe d'amélioration	3
2.2.1	Première main	3
2.2.2	Sécurité	4
2.2.3	Réseau	4
2.2.4	Code	5
2.2.5	Algorithmie et qualité du code	6

Chapitre 1

Introduction

Le projet sélectionné est “SoundStream”. Bien qu’assez abouti, le projet présente de nombreux axes d’amélioration.

Le projet répond déjà aux besoins proposés et va même au delà des nécessités, tout en proposant tout un système d’organisation. L’architecteure est robuste pour la maintenance de l’application (MVC).

Différents axes d’amélioration ont par ailleurs été sollicités par les référents du projet eux même.

Chapitre 2

Analyse

2.1 Fonctionnalités existantes

- Architecture Réseau plutôt manuelle
- Système de ticket
- Système d'organisation
- Page d'accueil différente pour chaque rôle (admin/commercial/marketing)
- Gestion des utilisateurs sur le site
- Organisation par calendrier
- Envoi un ticket qui sera géré par le back office en cas de mot de passe oublié

2.2 Axe d'amélioration

2.2.1 Première main

Le setup pour le projet est bien expliqué, fichier de requirements crée pour l'utilisateur, ce qui est parfait. En revanche un problème se présente à l'authentification : aucune information sur l'utilisateur donnée, ce qui rend la connexion au service complexe pour l'utilisateur lambda. Il y a nécessité d'analyser le code pour trouver le mot de passe ainsi que le nom d'utilisateur à utiliser. Il faudrait donc ajouter de la documentation afin de clarifier le processus de connexion (même si c'est 1234 il faut l'inscrire, par exemple dans le readme).

```
# 1. UTILISATEURS & ORGANISATIONS (Ton code existant)
password_clair = '12345'
hashed_pw = bcrypt.hashpw(password_clair.encode('utf-8'), bcrypt.gensalt()).decode('utf-8')

users = [
    ( "Romain", 'admin', hashed_pw),
    ( "Tristan", 'marketing', hashed_pw),
    ( "Abou", 'sales', hashed_pw)
]
```

Comment lancer l'application :

Il est nécessaire d'utiliser un environnement virtuel Python pour isoler les dépendances.

```
python3 -m venv env

# Windows
.\env\Scripts\activate

# macOS / Linux
source env/bin/activate

pip install -r requirements.txt

# Dans le répertoire SoundStream/Code
python3 main.py
```

Puis mettre l'URL <http://127.0.0.1:8000/> dan votre navigateur.

2.2.2 Sécurité

Le seeder est mal positionné dans le projet. Il demeure un outil pratique pour remplir les données en tant que développeur/installateur qui voudrait initialiser le projet mais cause une faille de sécurité car le mot de passe y est stocké en clair. De plus, le seeder donne exactement les mêmes codes pour les 3 utilisateurs générés (problème de redondance). Ce problème est aussi présent au niveau des l'enregistrement des lecteurs. Il faut donc modifier les problèmes de redondance, clarifier le readme et déplacer le seeder qui se trouve au plein milieu du code de production.

Une automatisation de la clé secrète (pour les cookies) serait convenu. L'idée serait de la stocker dans un fichier .env et d'également la vérifier.

Pour des raisons évidentes il faudrait que les administrateurs ne puissent s'auto supprimer.

```
players = [
    ('Abou', '10.100.27.134', 'OFFLINE', now, '12 Rue de Rivoli', '75000', 'Paris', 'centre commercial', 'aboubakry', 2), # JBL
]
```

2.2.3 Réseau

Axe principal d'amélioration du projet. Tout le système est manuel et dépend entièrement du site, le but est de retravailler l'architecture réseau est d'automatiser le tout , la gestion sera donc faite par l'admin à l'aide de tailscale qui lie les machines entre elles avec une adresse iP fixe. Elle sera mis automatiquement dans la base de donnée et entièrement manipulable (par ailleurs, la localisation ne sera plus arbitraire mais sera la vrai localisation de l'appareil).

Une sécurisation du protocole http pour le passer à https constituerait une amélioration considérable. Nous verrons pour distribuer l'application sur un serveur si le temps nous le permet.

2.2.4 Code

Plusieurs problèmes au niveau du code "localiser", comme lastsynchronization qui n'est pas lié à une requête SQL. Il faudrait mettre un CURRENTTIMESTAMP lors de l'initialisation de l'appareil afin de résoudre le problème.

Add a new player

Place Name :

claire

IP Address :

170.10.45.71

Physical Address :

25 rue de kenzo

City :

Paris

Postcode :

86150

Building :

Centre commercial

Username Linux:

Bonjour

Cancel

Save

ProgrammingError

sqlite3.ProgrammingError: Incorrect number of bindings supplied. The current statement uses 10, and there are 9 supplied.

Traceback (most recent call last)

```
File "home/bsosiane/ local/lib/python3.10/site-packages/flask/app.py", line 1536, in __call__
    return self.wsgi_app(environ, start_response)
File "home/bsosiane/ local/lib/python3.10/site-packages/flask/app.py", line 1514, in wsgi_app
    response = self.handle_exception(e)
File "home/bsosiane/ local/lib/python3.10/site-packages/flask/app.py", line 1511, in wsgi_app
    response = self.full_dispatch_request()
File "home/bsosiane/ local/lib/python3.10/site-packages/flask/app.py", line 919, in full_dispatch_request
    rv = self.handle_user_exception(e)
File "home/bsosiane/ local/lib/python3.10/site-packages/flask/app.py", line 917, in full_dispatch_request
    rv = self.dispatch_request()
File "home/bsosiane/ local/lib/python3.10/site-packages/flask/app.py", line 902, in dispatch_request
    return self.ensure_sync(self.view_functions[rule.endpoint])(**view_args) # type: ignore[no-any-return]
File "mntic/Users/sosia/Documents/project_cour/SoundStream/SoundStream/Code/app/controllers/LoginController.py", line 23, in decorated_function
    return f(*args, **kwargs)
File "mntic/Users/sosia/Documents/project_cour/SoundStream/SoundStream/Code/app/controllers/LoginController.py", line 38, in decorated_function
    return f(*args, **kwargs)
File "mntic/Users/sosia/Documents/project_cour/SoundStream/SoundStream/Code/app/controllers/DevicesController.py", line 212, in addPlayer
    sps.spdo.createDevice(
File "mntic/Users/sosia/Documents/project_cour/SoundStream/SoundStream/Code/app/models/SongPlayerDAO.py", line 30, in createDevice
    raise e
File "mntic/Users/sosia/Documents/project_cour/SoundStream/SoundStream/Code/app/models/SongPlayerDAO.py", line 26, in createDevice
    conn.execute(query, (name_place, ip_address, state, place_address, place_postcode, place_city, place_building_name, device_name, orga_id))
```

sqlite3.ProgrammingError: Incorrect number of bindings supplied. The current statement uses 10, and there are 9 supplied.

The debugger caught an exception in your WSGI application. You can now look at the traceback which led to the error.

To switch between the interactive traceback and the plaintext one, you can click on the "Traceback" headline. From the text traceback you can also create a paste of it. For code execution mouse-over the frame you want to debug and click on the console icon on the right side.

```

def createDevice(self, name_place, ip_address, state, place_address, place_postcode, place_city, place_building_name, device_name, orga_id) -> None:
    """ Create a new song player in the database. """
    conn = self._getDbConnection()
    try:
        query = '''
        INSERT INTO song_player
        (name_place, ip_address, state, last_synchronization, place_address, place_postcode, place_city, place_building_name, device_name, id_orga)
        VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?);
        '''
        conn.execute(query, (name_place, ip_address, state, place_address, place_postcode, place_city, place_building_name, device_name, orga_id))
        conn.commit()
    except Exception as e:
        conn.rollback()
        raise e
    finally:
        conn.close()

def addSongPlayerInDb(self, data_form_to_form) -> None:
    """ Add a new song player to the database. """
    conn = self._getDbConnection()
    try:
        query = '''
        INSERT INTO song_player
        (name_place, ip_address, state, last_synchronization, place_address, place_city, place_postcode, place_building_name, device_name, id_orga)
        VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?);
        '''
        conn.execute(query, data_form_to_form)
        conn.commit()
    except Exception as e:
        conn.rollback()
        raise e
    finally:

```

Il manque également des fonctions qui auraient été utiles, comme une redirection vers la page admin après la modification d'un lecteur (ce que le projet actuel ne fait pas).

2.2.5 Algorithmie et qualité du code

Une conversion du site en micro-service est prévue si le temps nous permet de réaliser cette opération de grande envergure.

Il y a aussi divers problèmes aux niveaux de fonctions rendues complexe inutilement. Un exemple serait que certaines fonctions interrogent la base de données et font plusieurs commits sans grand intérêt.

```

def deleteByUsername(self, username) -> None:
    """ Delete a user by username """
    conn = self._getDbConnection()

    # Delete the work_link entries first
    query = 'DELETE FROM work_link WHERE id_user = (SELECT id_user FROM user_ WHERE username = ?)'
    conn.execute(query, (username,))
    conn.commit()

    # Delete the user
    query = 'DELETE FROM user_ WHERE username = ?'
    conn.execute(query, (username,))
    conn.commit()
    conn.close()

```

Suppression bout par bout, il faut privilégier le DELETE en cascade,

```

    >Run | ☐Select
✓ CREATE TABLE IF NOT EXISTS work_link(
    id_user INT,
    id_orga INT,
    PRIMARY KEY(id_user, id_orga),
    FOREIGN KEY(id_user) REFERENCES user_(id_user),
    FOREIGN KEY(id_orga) REFERENCES organisation(id_orga)
);

    >Run | ☐Select
✓ CREATE TABLE IF NOT EXISTS composition(
    id_playlist INT,
    id_file INT,
    PRIMARY KEY(id_playlist, id_file),
    FOREIGN KEY(id_playlist) REFERENCES playlist(id_playlist),
    FOREIGN KEY(id_file) REFERENCES file(id_file)
);

    >Run | ☐Select
✓ CREATE TABLE IF NOT EXISTS interaction(
    id_playlist INT,
    id_user INT,
    PRIMARY KEY(id_playlist, id_user),
    FOREIGN KEY(id_playlist) REFERENCES playlist(id_playlist),
    FOREIGN KEY(id_user) REFERENCES user_(id_user)
);

    >Run | ☐Select
✓ CREATE TABLE IF NOT EXISTS planned(
    id_playlist INT,
    day_ VARCHAR(50),
    start_time TIME,
    PRIMARY KEY(id_playlist, day_),
    FOREIGN KEY(id_playlist) REFERENCES playlist(id_playlist),
    FOREIGN KEY(day_) REFERENCES Planning(day_)
);

```